SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr.A.JAYA MABEL RANI

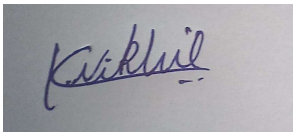
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	**5**
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	**5**
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	**5**
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	**5**
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	**5**
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	**5**
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	**5**

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	**5**
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINs and using appropriate indexes.	BL6 – Create	**5**
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	**5**

Code of Conduct:

*I **K.Nikhil / 192424366** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Question 1: Incorrect JOIN Syntax Causing Duplicate Results (5 Marks)

1. Given (Buggy) Query / Expression

```
SELECT e.emp_name, d.dept_name
FROM Employees e, Departments d, Projects p
WHERE e.dept_id = d.dept_id;
```

2. Problem Identification

- The query mixes old-style comma joins with a WHERE clause, but the table Projects (aliased p) is listed in the FROM clause without any join condition linking it to Employees or Departments.
- This leaves an unintended Cartesian product between (Employees ⋈ Departments) and the full Projects table.
- Result: every matching employee-department row is repeated once for every row in Projects, producing duplicate rows.

3. Debugging — Root Cause

Root cause: an extra table (Projects) was pulled into the FROM clause without a corresponding ON/WHERE condition, and the query uses implicit (comma) joins instead of explicit ANSI JOIN syntax, which makes such omissions easy to miss.

4. Corrected / Optimized Query

```
-- Corrected query: explicit JOIN syntax, unrelated table removed
SELECT e.emp_name, d.dept_name
FROM Employees e
JOIN Departments d
  ON e.dept_id = d.dept_id;
```

5. Analysis & Explanation

- Removed the unrelated Projects table since it was not needed for this result set.
- Replaced the implicit comma join with an explicit INNER JOIN ... ON, which forces every join condition to be stated and makes accidental Cartesian products far less likely.
- If employee-project data were genuinely required, the correct fix would be to join through the EmployeeProjects bridge table with proper ON conditions rather than joining Projects directly to Employees/Departments.

Question 2: Sub-query Missing Correlation (5 Marks)

1. Given (Buggy) Query / Expression

```
-- Intent: employees earning more than the AVERAGE salary of their OWN department
SELECT emp_name, dept_id, salary
FROM Employees e
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

2. Problem Identification

- The inner sub-query computes the average salary across the ENTIRE Employees table, not the average for each employee's own department.
- Because the sub-query never references the outer query's dept_id, it is an uncorrelated sub-query — it executes once and returns a single company-wide average, which is reused for every outer row.
- This gives the wrong comparison baseline for employees in departments whose average differs from the company-wide average.

3. Debugging — Root Cause

Root cause: the sub-query is missing the correlation condition (dept_id = e.dept_id) that ties each inner evaluation back to the current outer row.

4. Corrected / Optimized Query

```
-- Corrected query: correlated sub-query
SELECT emp_name, dept_id, salary
FROM Employees e
WHERE salary > (
   SELECT AVG(e2.salary)
   FROM Employees e2
   WHERE e2.dept_id = e.dept_id   -- correlation added
);
```

5. Analysis & Explanation

- Adding WHERE e2.dept_id = e.dept_id correlates the sub-query with the outer row, so the average is recalculated per department instead of once for the whole table.
- The sub-query alias e2 is used (instead of reusing e) to avoid ambiguity between inner and outer references.
- This is now logically correct: each employee is compared only against peers in the same department.

Question 3: Faulty SQL View Definition (5 Marks)

1. Given (Buggy) Query / Expression

```
CREATE VIEW DeptSummary AS
SELECT dept_id, emp_name, AVG(salary) AS avg_salary
FROM Employees
GROUP BY dept_id;
```

2. Problem Identification

- emp_name appears in the SELECT list but is neither part of the GROUP BY clause nor wrapped in an aggregate function.
- Under standard SQL (and strict modes such as MySQL's ONLY_FULL_GROUP_BY), this raises an error because emp_name has no single well-defined value per dept_id group — a department has many employees.
- Even where the engine silently allows it, the emp_name returned is arbitrary and misleading (a classic 'non-functionally-dependent column' bug).

3. Debugging — Root Cause

Root cause: the view mixes a detail-level column (emp_name) with an aggregated, group-level query without deciding the correct grain of the view.

4. Corrected / Optimized Query

```
-- Corrected and optimised view: one row per department (correct grain)
CREATE OR REPLACE VIEW DeptSummary AS
SELECT d.dept_id,
    d.dept_name,
    COUNT(e.emp_id)   AS employee_count,
    AVG(e.salary)     AS avg_salary
FROM Departments d
LEFT JOIN Employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

5. Analysis & Explanation

- Removed emp_name from the SELECT list and instead exposed COUNT(emp_id) so the view stays at the department grain with no functional-dependency violation.
- Used LEFT JOIN Departments -> Employees so departments with zero employees still appear (with employee_count = 0), instead of silently disappearing as they would with an INNER JOIN/plain GROUP BY on Employees alone.
- Optimization: an index on Employees(dept_id) (already implied by the foreign key) lets the join and GROUP BY use an index scan instead of a full table scan, keeping the view efficient as Employees grows.

Question 4: Relational Algebra Error in Natural Join (5 Marks)

1. Given (Buggy) Query / Expression

π(emp_name, project_name) (Employees ⋈ Projects)

2. Problem Identification

- Employees and Projects only share the attribute dept_id. A natural join (⋈) automatically joins on ALL identically named attributes.
- So this expression joins every employee to every project that belongs to the SAME department — it does not check whether the employee is actually assigned to that project.
- The result is logically incorrect: it returns (employee, project) pairs based on shared department, not on actual project assignment, which inflates and misrepresents the data (an unintended department-level Cartesian product disguised as a natural join).

3. Debugging — Root Cause

Root cause: the expression relies on an incidental shared attribute (dept_id) for a natural join, when the real relationship between Employees and Projects is expressed through the EmployeeProjects bridge relation, not a direct attribute match.

4. Corrected / Optimized Query

```
-- Corrected relational algebra: join through the bridge relation
π(emp_name, project_name) (
   Employees ⋈ EmployeeProjects ⋈ Projects
)

-- i.e. Employees.emp_id = EmployeeProjects.emp_id
--    EmployeeProjects.project_id = Projects.project_id
```

5. Analysis & Explanation

- Introducing EmployeeProjects restores the correct join path: Employees is joined to EmployeeProjects on emp_id, and that result is joined to Projects on project_id.
- Because emp_id and project_id are shared only where they are meant to represent the same real-world entity, the natural joins now compose correctly and return only genuine employee-to-project assignments.
- Equivalent SQL: SELECT e.emp_name, p.project_name FROM Employees e JOIN EmployeeProjects ep ON e.emp_id = ep.emp_id JOIN Projects p ON ep.project_id = p.project_id;

Question 5: Referential Integrity Violation (5 Marks)

1. Given (Buggy) Query / Expression

```
-- Departments table currently holds dept_id values 1-5 only
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id)
VALUES (105, 'Ashwin Kumar', 9, 55000, 101);
```

2. Problem Identification

- dept_id = 9 does not exist in the Departments table, so this INSERT creates an orphan row in Employees that references a non-existent parent.
- The fact that this statement executes without error indicates Employees.dept_id has no FOREIGN KEY constraint referencing Departments.dept_id — referential integrity is not being enforced at the schema level.

3. Debugging — Root Cause

Root cause: a missing FOREIGN KEY constraint on Employees.dept_id allows child rows to reference department IDs that do not exist in the parent table.

4. Corrected / Optimized Query

```
-- Step 1: enforce the constraint going forward
ALTER TABLE Employees
  ADD CONSTRAINT fk_emp_dept
  FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
  ON UPDATE CASCADE
  ON DELETE RESTRICT;

-- Step 2: correct the offending data before/while adding the constraint
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id)
VALUES (105, 'Ashwin Kumar', 3, 55000, 101);   -- valid dept_id
```

5. Analysis & Explanation

- The ALTER TABLE statement adds the missing foreign key so the database itself rejects any future INSERT/UPDATE that references a non-existent dept_id.
- ON DELETE RESTRICT prevents a department from being deleted while employees still reference it (protecting integrity from the parent side too); ON UPDATE CASCADE keeps dept_id values in sync if a department's key is ever changed.
- The corrected INSERT uses dept_id = 3, an ID that genuinely exists in Departments.

Question 6: Optimizing an Inefficient Nested-Loop Query into a JOIN (5 Marks)

1. Given (Buggy) Query / Expression

```
-- Find employees whose department is located in Chennai
SELECT e.emp_name
FROM Employees e
WHERE EXISTS (
   SELECT 1 FROM Departments d
   WHERE d.dept_id = e.dept_id
    AND d.location = 'Chennai'
);
```

2. Problem Identification

- The query is correct in result but inefficient: the correlated EXISTS sub-query is logically re-evaluated once for every row of Employees (a nested-loop pattern), instead of letting the optimizer use a single set-based join.
- On large tables this scales poorly, especially without supporting indexes.

3. Debugging — Root Cause

Root cause: row-by-row correlated sub-query execution instead of a set-based JOIN, which prevents the optimizer from choosing more efficient join algorithms (hash join / merge join).

4. Corrected / Optimized Query

```
-- Optimized: rewritten as a JOIN
SELECT DISTINCT e.emp_name
FROM Employees e
JOIN Departments d
  ON e.dept_id = d.dept_id
WHERE d.location = 'Chennai';

-- Supporting indexes
CREATE INDEX idx_employees_dept_id  ON Employees(dept_id);
CREATE INDEX idx_departments_loc    ON Departments(location);
```

5. Analysis & Explanation

- Converting EXISTS(...) into an explicit JOIN lets the query optimizer pick a single-pass join strategy (hash or merge join) instead of a nested loop re-executed per outer row.
- DISTINCT is added because a plain join could, in principle, duplicate an employee row if the underlying join condition ever matched more than once; EXISTS naturally de-duplicates, so this preserves the original semantics.
- The two indexes let the engine seek directly into Departments by location and support the join on dept_id, reducing the overall complexity from roughly O(n·m) row comparisons to close to O(n + m) with index lookups.

Question 7: Incorrect Aggregation in GROUP BY (5 Marks)

1. Given (Buggy) Query / Expression

```
SELECT emp_name, dept_id, SUM(salary) AS total_salary
FROM Employees
GROUP BY dept_id;
```

2. Problem Identification

- emp_name is selected alongside an aggregate (SUM) but is not included in GROUP BY and is not itself aggregated.
- Each group (one per dept_id) contains many employees, so emp_name has no single correct value per group — most engines either throw an error or (in permissive modes) silently pick an arbitrary emp_name per group, which is wrong and misleading.
- The aggregation itself is also ambiguous in intent: it's unclear whether the goal is a department-level total or a per-employee figure.

3. Debugging — Root Cause

Root cause: mismatched query grain — selecting a row-level column (emp_name) together with a group-level aggregate (SUM(salary)) grouped only by dept_id.

4. Corrected / Optimized Query

```
-- Option A: department-level total (drop emp_name, it doesn't belong at this grain)
SELECT dept_id, SUM(salary) AS total_salary
FROM Employees
GROUP BY dept_id;

-- Option B: keep per-employee detail AND show the department total, using a
window function
SELECT emp_name,
    dept_id,
    salary,
    SUM(salary) OVER (PARTITION BY dept_id) AS dept_total_salary
FROM Employees;
```

5. Analysis & Explanation

- Option A fixes the query by keeping it strictly at department grain, matching what GROUP BY dept_id can validly produce.
- Option B fixes it by keeping the original row-level grain (one row per employee, so emp_name is valid) while still exposing the department total via SUM(...) OVER (PARTITION BY dept_id), which is the correct SQL tool when both detail and aggregate values are needed together.
- Which option is 'correct' depends on the reporting requirement — this is the analysis step: identify the intended grain before choosing the fix.

Question 8: UPDATE Statement Unintentionally Modifying All Rows (5 Marks)

1. Given (Buggy) Query / Expression

```
-- Intent: give a 10% raise to employees in the IT department (dept_id = 3)
UPDATE Employees
SET salary = salary * 1.10;
```

2. Problem Identification

- The UPDATE statement has no WHERE clause, so it applies the 10% raise to every row in the Employees table, not just department 3.
- This is a classic and dangerous logical error — syntactically valid SQL that silently affects far more data than intended.

3. Debugging — Root Cause

Root cause: a missing WHERE clause that should scope the update to the intended department.

4. Corrected / Optimized Query

```
-- Corrected: scoped update
UPDATE Employees
SET salary = salary * 1.10
WHERE dept_id = 3;

-- Safe-update practice: verify the affected rows BEFORE running the UPDATE
SELECT emp_id, emp_name, salary, salary * 1.10 AS new_salary
FROM Employees
WHERE dept_id = 3;
```

5. Analysis & Explanation

- Adding WHERE dept_id = 3 restricts the raise to the intended department only.
- Best practice (code-quality/debugging discipline): run the equivalent SELECT first to preview exactly which rows and values will change, and wrap the UPDATE in a transaction (BEGIN ... COMMIT/ROLLBACK) so it can be undone if the affected row count looks wrong.

Question 9: Optimizing a Multi-Table Query with Nested Sub-queries (5 Marks)

1. Given (Buggy) Query / Expression

```
-- Find employees in the IT department working on the 'ERP Migration' project
SELECT emp_name
FROM Employees e
WHERE dept_id = (SELECT dept_id FROM Departments WHERE dept_name = 'IT')
  AND emp_id IN (
      SELECT emp_id FROM EmployeeProjects
      WHERE project_id = (
          SELECT project_id FROM Projects WHERE project_name = 'ERP
Migration'
      )
  );
```

2. Problem Identification

- The query nests three levels of sub-queries (scalar sub-query for dept_id, an IN sub-query, and a further nested scalar sub-query for project_id).
- Each scalar sub-query is safe only if it returns exactly one row; more importantly, the layered IN/scalar structure prevents the optimizer from planning this as a single set of joins and makes it harder to apply indexes across tables in one pass.
- This is functionally correct but inefficient and harder to read/maintain — exactly the kind of query this task asks to restructure.

3. Debugging — Root Cause

Root cause: the query is built as a chain of independent sub-queries rather than a single multi-table JOIN, forcing sequential, less-optimizable execution.

4. Corrected / Optimized Query

```
-- Optimized: restructured into JOINs
SELECT DISTINCT e.emp_name
FROM Employees e
JOIN Departments d       ON e.dept_id  = d.dept_id
JOIN EmployeeProjects ep ON e.emp_id   = ep.emp_id
JOIN Projects p          ON ep.project_id = p.project_id
WHERE d.dept_name = 'IT'
  AND p.project_name = 'ERP Migration';

-- Supporting indexes
CREATE INDEX idx_departments_name ON Departments(dept_name);
CREATE INDEX idx_projects_name    ON Projects(project_name);
CREATE INDEX idx_ep_emp_id        ON EmployeeProjects(emp_id);
CREATE INDEX idx_ep_project_id    ON EmployeeProjects(project_id);
```

5. Analysis & Explanation

- Restructuring into four explicit JOINs lets the query optimizer build one execution plan across all tables and choose efficient join orders/algorithms instead of resolving each sub-query independently.

- Indexing the lookup columns (dept_name, project_name) and the bridge table's foreign keys (emp_id, project_id) lets the engine seek directly to matching rows instead of scanning, which is the main optimization lever for this query.
- DISTINCT guards against duplicate rows if an employee were ever linked to the same project more than once in EmployeeProjects.

Question 10: Relational Calculus Error from Incorrect Quantifier Usage (5 Marks)

1. Given (Buggy) Query / Expression

```
-- Intent: employees who work on EVERY project belonging to their own department
{ e.emp_name | e ∈ Employees ∧
    ∃ p ( p ∈ Projects ∧ p.dept_id = e.dept_id ∧
        ∃ ep ( ep ∈ EmployeeProjects ∧
            ep.emp_id = e.emp_id ∧ ep.project_id = p.project_id ) ) }
```

2. Problem Identification

- The formula uses an existential quantifier (∃ p ...) to describe project coverage. ∃ only asserts that AT LEAST ONE matching project exists — it expresses 'works on some project of the department', not 'works on every project of the department'.
- Because the requirement is a universal condition ('all projects'), using ∃ instead of ∀ is a quantifier error. Depending on how the rest of the formula is negated when translated to SQL, this class of mistake commonly yields either the wrong (too-permissive) result set or, if paired with an incorrectly nested NOT EXISTS, an unintentionally empty result set.

3. Debugging — Root Cause

Root cause: the query needs a universal quantifier (∀) with an implication to express 'for every project of my department, I am assigned to it', but the formula instead uses an existential quantifier (∃), which only checks for one match.

4. Corrected / Optimized Query

```
-- Corrected relational calculus (universal quantification via implication)
{ e.emp_name | e ∈ Employees ∧
    ∀ p ( ¬(p ∈ Projects ∧ p.dept_id = e.dept_id) ∨
        ∃ ep ( ep ∈ EmployeeProjects ∧
            ep.emp_id = e.emp_id ∧ ep.project_id = p.project_id ) ) }

-- Equivalent SQL (standard 'division' pattern using double NOT EXISTS)
SELECT e.emp_name
FROM Employees e
WHERE NOT EXISTS (
   SELECT 1 FROM Projects p
   WHERE p.dept_id = e.dept_id
    AND NOT EXISTS (
        SELECT 1 FROM EmployeeProjects ep
        WHERE ep.emp_id = e.emp_id
          AND ep.project_id = p.project_id
    )
);
```

5. Analysis & Explanation

- The corrected calculus replaces ∃ p(...) with ∀ p(¬condition ∨ assigned), which is the standard way to express universal quantification: 'for every project p of my department, either p is not one of my department's projects, or I am assigned to p.'
- Because relational calculus/SQL do not have a native ∀, the equivalent SQL uses the classic double NOT EXISTS ('relational division') pattern: an employee qualifies only if there is NO project of their department for which there does NOT exist a matching assignment.
- This correctly returns employees who are assigned to every project in their own department, and both inner sub-queries are properly correlated to the outer employee (ep.emp_id = e.emp_id) and the department (p.dept_id = e.dept_id), avoiding the empty-result trap caused by missing correlation or the wrong quantifier.